提案書

koshi ｜ サーバーレス分散型 SNS 提案書

PC 一台で始める P2P 対話ボード、未踏ジュニア応募資料

game_ryo
v3.1.4 · 2026.10.18
MIT License · https://koshi.js.org

目次

01	提案概要	3
02	koshi はどんな分散型 SNS か	4
03	サーバーレス P2P アーキテクチャ	5
04	認証と暗号化	6
05	Nostr 連携	7
06	未踏ジュニアへの応募動機と適合性	8
07	開発スケジュール	9
08	期待されるインパクト	10
09	導入・操作ガイド	11
10	付録: API リファレンス、用語集、参考資料	12

01 · 提案概要

提案概要

本提案は、サーバーを立てずに PC 一台ですぐ始められる分散型 SNS「koshi」の開発計画である。集中型 SNS の検閲やアカウント凍結リスクを避け、ユーザーが自分のデータとアイデンティティを完全にコントロールできる仕組みを実現する。

核心 Takeaways

- サーバーレスモードがデフォルト。PostgreSQL や外部サーバーは不要で、SQLite だけで PC 一台から動く
- ed25519 鍵ベース認証と x25519 エンドツーエンド暗号化 DM で、投稿とプライベートメッセージを守る
- Nostr 互換（nsec/npub キー、リレー連携）で世界の分散型ソーシャルグラフと相互運用可能
- kb serve 一コマンドで REST API、WebSocket、Web UI、P2P メッシュが同時に立ち上がる

本ドキュメント回答の問題

サーバーを立てずに分散型 SNS をどう作るか？ ed25519 と x25519 でどう投稿と DM を守るか？ 未踏ジュニアの支援をどう活用して開発するか？

02 · KOSHI はどんな分散型 SNS か

koshi はどんな分散型 SNS か

koshi は投稿、フォロー、エンドツーエンド暗号化 DM、プロフィール編集、マルチアカウント管理、Nostr 連携、P2P メッシュ同期、Web UI を備えた分散型 SNS。すべて kb という単一 CLI コマンドから操作でき、サーバー構築を必要としない。

コア機能

koshi の核心機能は投稿、フォロー、エンドツーエンド暗号化 DM、プロフィール編集、マルチアカウント管理、Nostr 連携、P2P メッシュ同期、Web UI です。すべて kb コマンドから操作でき、専用の管理画面やサーバー構築を必要としません。

分散認証

認証は ed25519 公開鍵暗号に基づき、ユーザーは自身のキーペアを保有します。パスワードベースの集中管理ではなく、鍵署名による本人確認を行い、アカウントの所有権をユーザー自身が握ります。

P2P データ同期

投稿と DM は hypercore の追記専用ログに追記され、hyperswarm による P2P トピックを通じて接続ピア間に自動複製されます。サーバーを介さずにデータがメッシュ状に同期されます。

エンドツーエンド暗号化 DM とチャット

DM は x25519 でエンドツーエンド暗号化され、送信者と受信者のみが復号できます。WebSocket を用いた LINE 風バブルチャット UI をターミナル上で提供し、リアルタイム対話を可能にします。

03 · サーバーレス P2P アーキテクチャ

サーバーレス P2P アーキテクチャ

koshi は SQLite ローカルデータベースと hypercore/corestore/hyperswarm による P2P メッシュを組み合わせたサーバーレスアーキテクチャを採用。外部データベースやサーバー構築を一切必要とせず、家庭の PC 一台で動作する。

サーバーレスモード（デフォルト）

サーバーレスモードでは、すべてのデータは ~/.config/koshi/local.db の SQLite に保存されます。外部データベースは不要で、npm install -g @ryopc/koshi 後の kb register <名前> ですぐに利用を開始できます。

P2P メッシュ層

P2P 層は hypercore（追記専用ログ）、corestore（複数の hypercore インスタンスの管理）、hyperswarm（P2P ネットワーク発見と接続管理）で構成されます。グローバルボードトピックに投稿とプロフィール発表がゴシップ配信されます。

P2P ブリッジとリモートモード

P2P ブリッジが P2P のデータとローカル SQLite を整合させ、Web UI や CLI は SQLite 経由でデータを読み書きします。P2P は Node.js 環境で動作し、ブラウザは REST API と WebSocket 経由でローカルノードにアクセスします。

リモートモードでは PostgreSQL バックエンドを持つ koshi サーバーに接続でき、kb connect <URL> で切り替えられます。まずは PC 一台のサーバーレスから始め、後で拡張できます。

図 1: koshi のサーバーレス P2P アーキテクチャ。CLI と SQLite、P2P メッシュがローカルPCで動き、ブラウザは WebSocket で繋がる。

04 · 認証と暗号化

認証と暗号化

koshi は ed25519 署名で投稿・DM のなりすましや改ざんを防ぎ、x25519 でダイレクトメッセージをエンドツーエンド暗号化する。送信者と受信者だけが読め、途中の誰にも中身はわからない。

認証層と署名

インターネットでメッセージを送るとき、途中の誰かに中身を見られたくないものです。koshi は ed25519 を使い、すべての投稿と DM にデジタル署名を付けることで、本当にその人が書いたものかを確認し、改ざんを防ぎます。

エンドツーエンド暗号化 DM

ダイレクトメッセージは x25519 でエンドツーエンド暗号化されます。送信者と受信者だけが鍵を持っており、途中でデータを傍受されても中身は読めません。ターミナル上のバブルチャット UI でリアルタイムに暗号化チャットができます。

JWT 認証とレート制限

認証には JWT というチケットを使い、24 時間で失効します。なりすましや不正アクセスを防ぐため、認証画面には 1 分間に 10 回までの制限を設け、セキュリティヘッダや SQL インジェクション対策も施しています。

設定保護

設定ファイルのアクセス権は 0600 に設定され、所有者のみが読み書きできます。鍵や認証情報が他の人から見られないよう保護されています。

05 · NOSTR 連携

Nostr 連携

koshi は Nostr プロトコル（nsec/npub 鍵、中継所＝リレー）と互換。kb nostr push で koshi の投稿を世界の Nostr リレーに流し、kb nostr pull で世界の投稿を koshi のフィードに取り込める。

Nostr とは

Nostr は、世界中に点在する中継所（リレー）を通じて投稿をやり取りする分散型プロトコルです。ユーザーは nsec と npub という鍵ペアを持ち、どのサービスにも縛られません。

クロス投稿とプル

koshi は kb nostr key generate で Nostr の鍵を作れ、kb nostr relay add で中継所を登録できます。kb nostr push で koshi の投稿を Nostr のリレーに流せば、世界中の Nostr ユーザーに届きます。

逆に kb nostr pull で Nostr の投稿を koshi のフィードに取り込み、koshi のタイムラインと合流できます。koshi と Nostr の両方の世界を一つの画面で楽しめます。

発信の場として

オープンソースの分散型ソーシャルグラフの一部になれるため、自分の作品を世界に発信したいクリエイターにとって強力な手段になります。

06 · 未踏ジュニアへの応募動機と適合性

未踏ジュニアへの応募動機と適合性

koshi は未踏ジュニアの支援内容（メンター、最大50万円の開発資金、開発場所・機材の提供、参加費無料）と親和性が高く、サーバーレスでサーバー代がかからないため、開発資金を機材や学習に充てられる。MIT ライセンスのオープンソースで成果を公開しやすい。

未踏ジュニアの支援内容

未踏ジュニアは、独創的なアイデアと技術を持つクリエイターを支援するプログラムです。採択されると、経験豊富なメンターからのアドバイスが受けられ、プログラミングの質問やプロジェクトの方針を相談できます。

開発資金と環境

各グループに最大50万円の開発資金が支援され、開発場所や機材も提供されます。参加費は無料で、イベントへの交通費・宿泊費も支給されます。開発期間は約6ケ月間です。

サーバーレスのメリット

koshi はサーバーレスなのでサーバー代がかからず、家庭の PC と無料のツールだけで開発を始められます。50万円の開発資金は、サーバー代ではなく開発機材や有料 API、学習教材などに使えます。

オープンソースと公開

MIT ライセンスのオープンソースなので、開発したコードを GitHub で公開し、成果報告会で発表しやすいです。特に顕著な成果を残すと、スーパークリエータとして認定される道も開けます。

理念との合致

サーバー不要・オープンソース・P2P という koshi の特徴は、クリエイターが自分のデータを自分で管理し、表現の場を持つことを支援する未踏ジュニアの理念と合致しています。

07 · 開発スケジュール

開発スケジュール

未踏ジュニアの開発期間（約6ケ月）に合わせたロードマップ。まずサーバーレスの基本機能を固め、次に Nostr 連携と Web UI を拡充し、最後に P2P メッシュの安定化とマルチアカウントを完成させる。

1ケ月目

koshi のインストールと kb register、kb post、kb feed の基本動作確認。ed25519 認証と SQLite ローカル DB の動作を安定させる。

2ケ月目

kb serve で Web UI と WebSocket リアルタイム通信を動かし、ブラウザからの投稿・DM を確認。kb chat のバブルチャット UI を使いやすくする。

3ケ月目

Nostr 連携を実装。kb nostr key generate、kb nostr push、kb nostr pull で世界の Nostr ネットワークと繋げる。

4ケ月目

P2P メッシュ（hypercore + hyperswarm）の安定化。複数の PC で投稿が同期されることを確認し、マルチアカウント切り替え（kb switch）を完成させる。

5ケ月目

5ケ月目: セキュリテイ見直し（レート制限、署名検証、暗号化 DM の動作確認）とドキュメント整備。未踏ジュニアのメンターと相談しながら品質を高める。

6ケ月目

6ケ月目: koshi で実際に投稿・DM を行い、成果報告会でデモ発表。未踏ジュニアのスーパークリエータ認定を目指します。

08 · 期待されるインパクト

期待されるインパクト

koshi はサーバー不要の分散型 SNS として、クリエイターが自分のデータを自分で管理し、検閲に強い表現の場を持つことを実現する。MIT ライセンスによりコミュニティによる改良とフォークを歓迎し、未来の分散型ソーシャルの基盤になりうる。

データ主権の実現

koshi がもたらす最大のインパクトは、自分のデータは自分で管理するという体験を、サーバー知識がなくても始められることです。分散型 SNS のハードルを下げ、より多くの人が自分の声を発信できる場を作ります。

フォークと改良

MIT ライセンスのオープンソースなので、誰でも koshi をフォークして自分だけの SNS を作れます。クリエイターが koshi をベースに新機能を加え、世界に公開することも可能です。

オフライン対応とレジリエンス

P2P メッシュが安定すれば、インターネット接続が限定的な環境でも、近くの仲間と直接データをやり取りできます。災害時やイベント会場など、サーバーが使えない場面でも役立ちます。

ロードマップ

ロードマップは P2P メッシュのさらなる安定化、Web UI の機能拡充、モバイル端末からのアクセス改善、より大規模なメッシュネットワークへの対応です。未踏ジュニアをきっかけに、分散型ソーシャルを支える技術者が育つことを期待します。

09 · 導入・操作ガイド

導入・操作ガイド

koshi の導入は npm install -g @ryopc/koshi と kb register <名前> のニステップで完了。kb serve でローカルノード（REST API、WebSocket、Web UI、P2P メッシュ）が同時に立ち上がり、ブラウザからも操作できる。

インストール

まずは Node.js 20.0.0 以上をインストールします。次にターミナルで npm install -g @ryopc/koshi と打つと、kb というコマンドが使えるようになります。

アカウント作成

kb register ＜好きな名前＞ で自分のアカウントを作ります。ed25519 の鍵ペアが自動生成され、設定ファイルに保存されます。これであなた専用の分散型 SNS アカウントができました。

投稿とフォロー

kb post Hello koshi と打つと、最初の投稿が作成され、kb feed でタイムラインを見られます。kb follow で友達をフォローし、kb dm で暗号化されたダイレクトメッセージを送れます。

ローカルサーバーと Web UI

kb serve と打つと、PC 上で koshi サーバーが立ち上がり、http://localhost:3000 にブラウザでアクセスできます。REST API、WebSocket、Web UI が全部使え、ブラウザからも投稿や DM を確認できます。

リモート接続と停止

リモートサーバーに繋ぎたいときは kb connect ＜URL＞ と打つだけ。kb connect --reset で PC 一台のサーバーレスに戻せます。kb serve --daemon でバックグラウンド実行、kb serve --stop で停止も簡単です。

10 · 付録

付録: API リファレンス、用語集、参考資料

付録では koshi の REST API エンドポイント一覧、主要用語の定義、参考となる外部資料をまとめる。実装者や運用者が koshi を統合・拡張する際の参照資料とする。

A. REST API エンドポイント

REST API は認証(/api/auth/register、/api/auth/login)、ユーザー(/api/users/:username、/api/users/me、フォロー関連)、投稿(/api/posts/feed、/api/posts、/api/posts/:id)、DM(/api/dms、/api/dms/:userId、既読マーク、未読数)、管理(/api/admin/users 等)、P2P(/api/p2p/status、/api/p2p/peers、/api/p2p/posts、/api/p2p/sync 等)、WebSocket(/ws?token={jwt})、ヘルスチェック(/api/health)を提供する。

B. 用語集

ed25519: 公開鍵署名アルゴリズム。認証と署名に使用。

x25519: 鍵合意アルゴリズム。DM 暗号化に使用。

hypercore: 追加専用ログ。

corestore: hypercore ファクトリ。

hyperswarm: P2P 発見・接続管理。

Nostr: 分散型ソーシャルプロトコル。nsec/npub キー。

JWT: 認証トークン。24 時間有効。

SQLite: ファイルベースのデータベース。

C. 参考資料

- koshi README.md (v3.1.4) - https://github.com/ryopc/koshi
- 未踏ジュニア公式サイト - https://jr.mitou.org/
- @noble/ed25519 - https://github.com/paulmillr/noble-ed25519
- tweetnacl - https://tweetnacl.js.org/
- ed2curve - https://github.com/dominictarr/ed2curve
- hypercore / corestore / hyperswarm - https://github.com/hypercore-protocol
- nostr-tools - https://github.com/paulmillr/nostr-tools
- better-sqlite3 - https://github.com/WiseLibs/better-sqlite3
- Express / Helmet - https://expressjs.com/ / https://helmetjs.github.io/